



@qa_insights

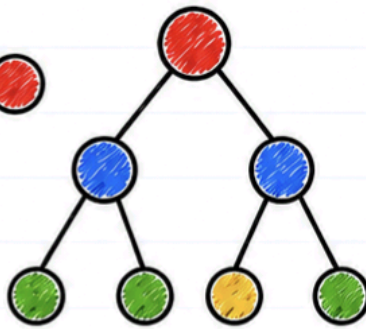
Part-1

DSA

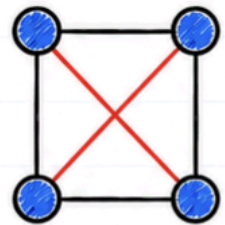
COMPLETE NOTES

(Data Structures & Algorithms)

Part-1

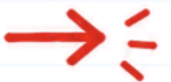


0	
1	
2	
3	



BEGINNER TO ADVANCED

SWIPE TO NEXT

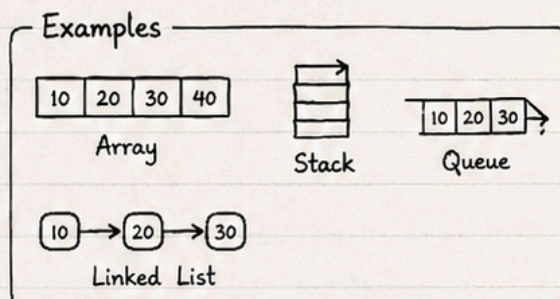


Qa_insights better careers

Introduction to Data Structures & Algorithms (DSA)

① What is Data Structure?

- A Data Structure is a way of organizing and storing data in a computer so that it can be used efficiently.
- It helps us perform operations like insertion, deletion, searching, etc. in optimized time.



② What is Algorithm?

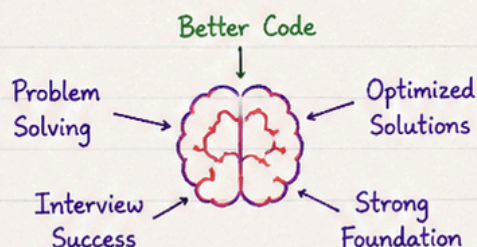
- An Algorithm is a step-by-step procedure or a set of rules to solve a problem or perform a task.
- It is a finite sequence of well-defined instructions.

Example : Find Maximum in Array

1. Start
2. Assume first element as max
3. Compare with remaining elements
4. Update max if any element is greater
5. Return max
6. Stop

③ Why DSA is Important?

- ✓ Helps in writing optimized and efficient code.
- ✓ Improves problem-solving and logical thinking.
- ✓ Reduces time and memory complexity.
- ✓ Essential for technical interviews.
- ✓ Builds strong foundation for advanced topics.



④ Real-world Examples

- GPS Navigation → Graphs (Shortest Path)
- E-commerce → Searching, Sorting
- File Systems → Trees
- Browser History → Stack
- Music Playlist → Queue

⑤ Where DSA is Used?

- Software Development → Efficient code, System design, Scalability
- Artificial Intelligence → ML models, Data processing, Optimization
- Games → Path finding, Collision detection, AI for NPCs
- Web Development → Searching, Caching, Indexing, Performance
- Technical Interviews → Core part of almost all coding rounds

★ Remember:

Good knowledge of DSA helps you write better code, solve hard problems and build efficient systems.

Time & Space Complexity

① What is Time Complexity?

Time Complexity is the amount of time an algorithm takes to run as a function of input size (n).

☆ Measures how fast an algorithm runs.

② What is Space Complexity?

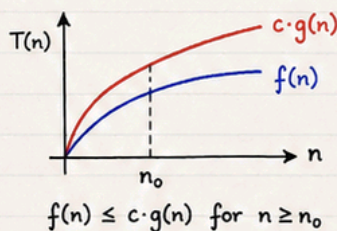
Space Complexity is the amount of memory an algorithm uses as a function of input size (n).

☆ Measures how much extra memory an algorithm needs.

③ Asymptotic Notations

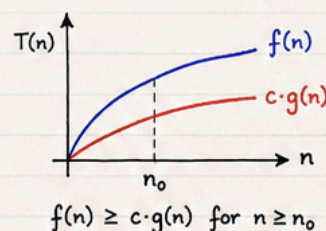
Big O (O)

Upper bound : Worst-case Complexity. Gives the maximum time an algorithm can take.



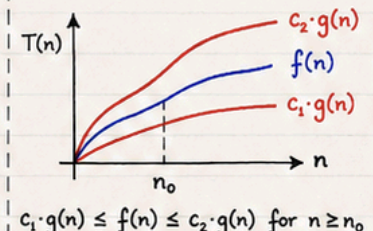
Big Ω (Omega)

Lower bound : Best-case Complexity. Gives the minimum time an algorithm can take.



Big Θ (Theta)

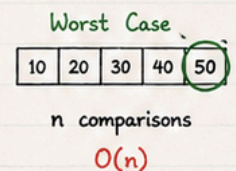
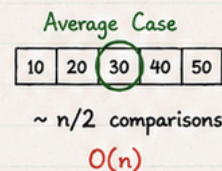
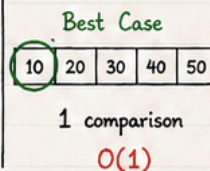
Tight bound : Average-case Complexity. Gives the exact growth rate.



④ Best, Average & Worst Case

- **Best Case** : Minimum time for an input.
- **Average Case** : Average time over all inputs.
- **Worst Case** : Maximum time for an input.

Example : Linear Search in Array of size n



⑤ Complexity Cheat Sheet

Growth Rate	Notation	Name	Time Complexity	Space Complexity	Example
1	$O(1)$	Constant	Best	Constant	Access array element
$\log n$	$O(\log n)$	Logarithmic	Very Good	Logarithmic	Binary Search
n	$O(n)$	Linear	Good	Linear	Linear Search, Traversal
$n \log n$	$O(n \log n)$	Linearithmic	Better	Linearithmic	Merge Sort, Heap Sort
n^2	$O(n^2)$	Quadratic	Bad	Quadratic	Bubble Sort, Selection Sort
2^n	$O(2^n)$	Exponential	Very Bad	Exponential	Recursive subsets
$n!$	$O(n!)$	Factorial	Worst	Factorial	Permutations (Brute Force)

💡 Key Takeaways

- ✓ Always aim for lower time and space complexity.
- ✓ Focus on worst-case while analyzing.
- ✓ Choose the right data structure + algorithm.

☆ Remember

Good code works, but optimized code scales!

Arrays

① Definition

- An Array is a linear data structure that stores elements of the **same data type** in **contiguous memory** locations.

② Memory Representation

- Elements are stored in **continuous memory** and accessed using an **index**.

Index →	0	1	2	3	4
arr →	10	20	30	40	50
Address →	1000	1004	1008	1012	1016

(Assuming 4 bytes per integer)

③ Operations

- Traversal** : Visit each element.
- Insertion** : Add element at a position.
- Deletion** : Remove element from a position.
- Access** : Access element using index.
- Search** : Find element by value.



Important Note

- Array size must be fixed in most languages.
- Indexing usually starts from 0.

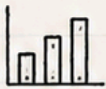
④ Advantages

- ✓ Random access in $O(1)$ time.
- ✓ Simple to understand and implement.
- ✓ Cache friendly due to contiguous memory allocation.

⑤ Disadvantages

- ✗ Fixed size (cannot grow/shrink easily).
- ✗ Insertion and deletion can be costly.
- ✗ Requires contiguous memory block.

⑥ Applications



Store list of items/scores



Matrices and table representation



Implementing other data structures (Stack, Queue, Heap)



Searching (Linear/Binary Search)



Frequency counting and hashing

⑦ Time Complexity

Operation	Best Case	Average Case	Worst Case	Space Complexity
Access	$O(1)$	$O(1)$	$O(1)$	$O(n)$
Search (Unsorted)	$O(1)^*$	$O(n)$	$O(n)$	$O(1)$
Search (Sorted)	$O(1)^*$	$O(\log n)$	$O(\log n)$	$O(1)$
Insertion	$O(1)$ (at end)	$O(n)$	$O(n)$	$O(1)$
Deletion	$O(1)$ (at end)	$O(n)$	$O(n)$	$O(1)$
Traversal	$O(n)$	$O(n)$	$O(n)$	$O(1)$

* If the first element is the target.

★ Remember

Arrays are the building block of many data structures and algorithms.

Strings

① What is a String?

- A String is a sequence of characters used to represent text.
- In most languages, strings are stored as an array of characters.
- Example: "hello"

Index →	0	1	2	3	4
char →	'h'	'e'	'l'	'l'	'o'

Length = 5

@qa_insights

② String Operations

- Access** : `s.charAt(i)` → $O(1)$
- Length** : `s.length()` → $O(1)$
- Concatenation** : `s1 + s2` / `s1.concat(s2)`
- Substring** : `s.substring(l, r)` → $O(n)$
- Comparison** : `s1.equals(s2)` → $O(n)$
- Search** : `s.indexOf(c/str)` → $O(n)$
- Replace** : `s.replace(old, new)` → $O(n)$
- Split** : `s.split("delim")` → $O(n)$

@qa_insights

③ Immutable vs Mutable Strings

Immutable (e.g., Java String)	Mutable (e.g., StringBuilder)
- Cannot be changed after creation. - Any modification creates a new object. - Stored in String Pool. - Example: <code>String s = "abc"; s = s + "d";</code> // new object	- Can be modified after creation. - Changes happen in the same object. - Not stored in String Pool. - Example: <code>StringBuilder sb = new StringBuilder("abc"); sb.append("d");</code> // "abcd"

④ Common Interview Questions

- Given a string, check if it is a palindrome.
- Reverse a string.
- Check if two strings are anagrams.
- Find the first non-repeating character.
- Longest common prefix.
- Implement string to integer (atoi).
- Count vowels and consonants.
- Check if a string is rotation of another.



⑤ Time Complexity of Common String Operations

@qa_insights

Operation	Description	Time Complexity		
		Best Case	Average Case	Worst Case
Access (<code>s.charAt(i)</code>)	Access character at index i	$O(1)$	$O(1)$	$O(1)$
Length (<code>s.length()</code>)	Get length of string	$O(1)$	$O(1)$	$O(1)$
Concatenation (<code>s1 + s2</code>)	Join two strings	$O(n)$	$O(n)$	$O(n)$
Substring (<code>s.substring(l, r)</code>)	Extract substring	$O(1)^*$	$O(n)$	$O(n)$
Comparison (<code>s1.equals(s2)</code>)	Compare two strings	$O(1)^*$	$O(n)$	$O(n)$
Search (<code>s.indexOf(c/str)</code>)	Find character / substring	$O(1)^*$	$O(n)$	$O(n)$
Replace (<code>s.replace(a, b)</code>)	Replace occurrences	$O(n)$	$O(n)$	$O(n)$
Split (<code>s.split(delim)</code>)	Split by delimiter	$O(n)$	$O(n)$	$O(n)$

* Best case when the first character matches / found at first position.

@qa_insights

@qa_insights

★ Remember

Strings are heavily used in DSA problems. Choose the right string type based on mutability and performance needs.

✓ Interview Tip

✓ Practice pattern based string problems. Most interview questions use strings in some form!

String = Char Array
+
Powerful Operations

Searching Algorithms

① Linear Search

- Linear Search checks each element one by one from start to end until the **target** is found.

Example

Arr =

7	2	9	4	1
---	---	---	---	---

 Target = 4
Index 0 1 2 3 4

Step

$\overset{1}{[7\ 2\ 9\ 4\ 1]}$ $\overset{2}{[7\ 2\ 9\ 4\ 1]}$ $\overset{3}{[7\ 2\ 9\ 4\ 1]}$ $\overset{4}{[7\ 2\ 9\ 4\ 1]}$
 $\uparrow$ $\uparrow$ $\uparrow$ $\uparrow$
 $7 \neq 4$ $2 \neq 4$ $9 \neq 4$ $4 = 4$
Found at index 3

② Binary Search (Array must be sorted)

- Binary Search repeatedly divides the search interval in half.

Example

Arr =

1	3	5	7	9	11	13
---	---	---	---	---	----	----

 Target = 7
Index 0 1 2 3 4 5 6

Steps

- low = 0, high = 6

1	3	5	7	9	11	13
---	---	---	---	---	----	----

 $\uparrow$
mid = 3

arr[mid] = 7
Found at index 3

③ Comparison

@qa_insights

	Linear Search	Binary Search
Works on	Unsorted or Sorted	Sorted Only
Approach	Check one by one	Divide & Conquer
Efficiency	Slower ($O(n)$)	Faster ($O(\log n)$)
Best Use	Small / Unsorted data	Large / Sorted data
Data Requirement	No order needed	Requires sorted data

④ When to Use?

@qa_insights

Use Linear Search When:

- ✓ Data is unsorted.
- ✓ Dataset size is small.
- ✓ Simplicity is more important.



Use Binary Search When:

- ✓ Data is sorted.
- ✓ Dataset size is large.
- ✓ You need faster search ($O(\log n)$).



⑤ Time & Space Complexity

@qa_insights

Algorithm	Best Case	Average Case	Worst Case	Space Complexity	Notes
Linear Search	$O(1)$	$O(n)$	$O(n)$	$O(1)$	Best case: first element is target.
Binary Search	$O(1)$	$O(\log n)$	$O(\log n)$	$O(1)$	Requires sorted array. Divide by 2 each step.



Important Note

Binary Search is much faster, but it only works on sorted data. **Sorting + Binary Search** is a powerful combination!

@qa_insights

★ Interview Tip

Always check if the array is sorted.
If not sorted → use Linear Search.
If sorted → use Binary Search.

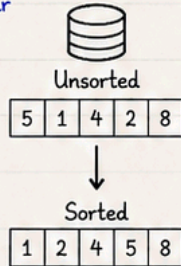


@qa_insights

Sorting Algorithms

① Why Sorting?

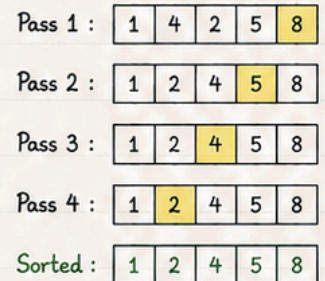
- Sorting arranges data in a particular order (**ascending / descending**).
- It helps in faster searching, easy comparison and better data analysis.
- Many advanced algorithms (like **Binary Search**, **DP**, **Greedy**) require sorted data.



② Bubble Sort

- Compare adjacent elements and swap if they are in wrong order.
- Pushes largest element to the end in each pass.

Example : Sort [5, 1, 4, 2, 8]



③ Selection Sort

- Find the minimum element from unsorted part and place it at the beginning.
- In each pass, the smallest element comes to its correct position.

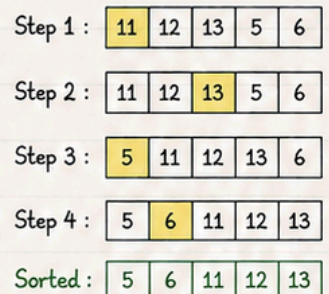
Example : Sort [64, 25, 12, 22, 11]



④ Insertion Sort

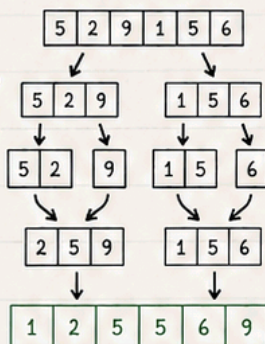
- Builds the sorted array one element at a time.
- Pick element and insert it into its correct position in the sorted part.

Example : Sort [12, 11, 13, 5, 6]



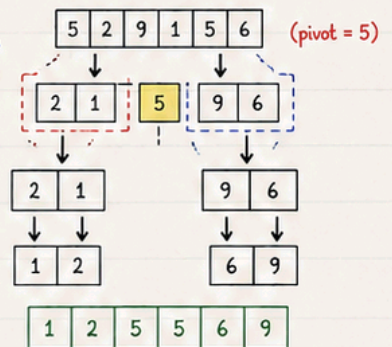
⑤ Merge Sort

- Divide the array into two halves, sort each half and merge them.
- Follows Divide & Conquer.
- Stable and works well for linked lists and large data.



⑥ Quick Sort

- Pick a pivot and partition the array into two parts: less than pivot and greater than pivot.
- Recursively apply on subarrays.
- Average case is very fast.



⑦ Complexity Comparison

Algorithm	Time Complexity			Space Complexity	Stable	In-place
	Best Case	Average Case	Worst Case			
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes	Yes
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	No	Yes
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes	Yes
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes	No
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)^*$	No	Yes

* Recursive stack space in average case.

Key Takeaways

- ✓ Use Insertion Sort for small or nearly sorted data.
- ✓ Use Merge Sort for stable sorting and large datasets.
- ✓ Use Quick Sort for in-place sorting with best average performance.

Interview Tip

Always mention best, average and worst case complexity in interviews.

Recursion

① What is Recursion?

- Recursion is a technique where a function calls itself to solve smaller instances of the **same problem**.
- It breaks a complex problem into simpler subproblems until it reaches a **stopping condition**.

Example: Factorial of n
 $\text{fact}(n) = n \times \text{fact}(n-1)$
 $\text{fact}(0) = 1$ (stopping condition)

② Base Case

- The condition that **stops** the recursion.
- Without base case, recursion will run infinitely.

Example:

$\text{fact}(0) = 1$
 or
 $\text{fact}(1) = 1$

③ Recursive Case

- The part where the function calls itself with a **smaller input**.
- This moves the solution towards the base case.

Example:

$\text{fact}(n) = n \times \text{fact}(n-1)$
 $(n > 1)$

④ Call Stack

- Every recursive call is pushed onto the **Call Stack**.
- When the base case is reached, the calls return in **reverse order (LIFO)**.

@qa_insights

Example: $\text{fact}(4)$

Function Calls (Push)

$\text{fact}(4)$
 $\downarrow$
 $\text{fact}(3)$
 $\downarrow$
 $\text{fact}(2)$
 $\downarrow$
 $\text{fact}(1)$
 $\downarrow$
 $\text{fact}(0)$

Call Stack

$\text{fact}(4)$
$\text{fact}(3)$
$\text{fact}(2)$
$\text{fact}(1)$
$\text{fact}(0)$

Returns (Pop)

24
 $\uparrow$
 6
 $\uparrow$
 2
 $\uparrow$
 1
 $\uparrow$
 1



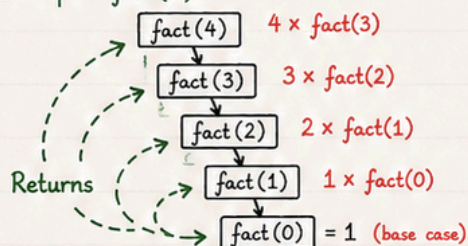
Important Note

Recursion uses extra memory in the stack. Too deep recursion can cause **Stack Overflow**.

⑤ Recursion Tree

- Shows how the problem is broken into subproblems.

Example: $\text{fact}(4)$



⑥ Advantages & Drawbacks

Advantages

- ✓ Simplifies complex problems.
- ✓ Easy to write and understand.
- ✓ Natural for problems with repeated substructure.
- ✓ Useful for tree/graph traversals, backtracking, divide & conquer.

Drawbacks

- ✗ High memory usage (call stack).
- ✗ Slower due to function call overhead.
- ✗ Can cause Stack Overflow for deep recursion.

★ Interview Tip

Always identify the base case first, then write the recursive case.
 Dry run with small values to understand the flow!



Common Mistakes

- Missing or wrong base case.
- Not reducing the problem size.
- Infinite recursion.



Key Takeaway

Recursion = Solve **small** subproblems + **Combine** the results.

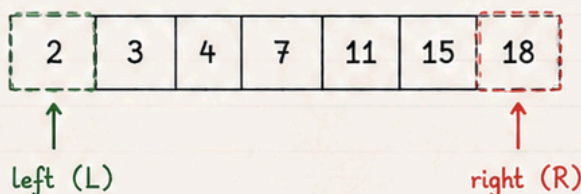
Two Pointer Technique

① Concept

- Two Pointer Technique uses two indices that move through the data structure to solve problems efficiently.
- It **reduces time complexity** by avoiding extra loops.
- Mostly used on **sorted arrays** or linear traversals.

@qa_insights

② Left & Right Pointer



L starts from beginning (0)

R starts from end (n-1)

How it works?

- Move L or R based on the condition.
- Narrow down the search space.

③ Use Cases

- ✓ Finding **pairs** with a given **sum** in sorted array.
- ✓ Removing **duplicates** from sorted array.
- ✓ Checking **palindrome** in string.
- ✓ Container With Most Water problem.
- ✓ Trapping Rain Water variations.

④ Sample Problems

- 1. **Two Sum II** - Input array is sorted
- 2. **Remove Duplicates from Sorted Array**
- 3. **Valid Palindrome**
- 4. **3Sum** - Find all unique triplets that sum to zero
- 5. **Container With Most Water**

@qa_insights

⑤ Complexity

Approach	Time Complexity	Space Complexity	Notes
Two Pointer	$O(n)$	$O(1)$	Each pointer moves at most n times.
Brute Force	$O(n^2)$	$O(1)$	Uses nested loops.



Important Note

Two Pointer works best when the data is sorted or can be traversed sequentially.

★ Interview Tip

Always check if the problem can be solved in $O(n)$ using two pointers before trying complex solutions.



Common Mistakes

- ✗ Forgetting to move pointers in some cases.
- ✗ Incorrect pointer movement (skipping elements).
- ✗ Not handling edge cases (empty array, single element).

@qa_insights



Key Takeaway: Two Pointer Technique is simple but powerful. It helps reduce time complexity from $O(n^2)$ to $O(n)$ in many problems.

@qa_insights

@qa_insights | Quality Insights, Better Careers!

@qa_insights

Sliding Window

① Fixed Window

- Window size (k) is **fixed**.
- We slide the window by one step at a time.
- Useful when we need subarrays/substrings of exact size k .

Example :

Array = [1, 3, 2, 6, 4, 1, 8], $k = 3$

1	3	2	6	4	1	8	Sum = 6
0	1	2	3	4	5	6	
1	3	2	6	4	1	8	Sum = 11
1	3	2	6	4	1	8	Sum = 12
1	3	2	6	4	1	8	Sum = 11
1	3	2	6	4	1	8	Sum = 13

@qa_insights

② Variable Window

- Window size is **not fixed**.
- Window can expand or shrink based on the condition.
- Useful to find smallest / longest subarray/substring satisfying a condition.

Example (Longest subarray with sum $\leq K$)

Array = [2, 1, 5, 2, 3, 2], $K = 7$

2	1	5	2	3	2
0	1	2	3	4	5
L					R

Expand R to increase sum.
If sum > K, move L to shrink window.

③ Applications

- ✓ Subarray / substring problems
- ✓ Cumulative problems (sum, product, count)
- ✓ Finding longest / shortest subarray
- ✓ Used in streaming data problems
- ✓ Used in many interview questions!

@qa_insights

④ Common Problems

1. Maximum Sum Subarray of Size K
2. Smallest Subarray with Sum $\geq K$
3. Longest Substring Without Repeating Characters
4. Longest Substring with At Most K Distinct Characters
5. Fruit Into Baskets (at most 2 types of fruits)
6. Minimum Window Substring
7. Max Consecutive Ones III (flip at most K zeros)



Interview Tip

Identify if the problem asks for contiguous subarray / substring.

If yes → Think Sliding Window!

⑤ Complexity

Let n = size of array/string

- Each element is visited at most twice (by L or R).
- So overall complexity is **linear**.

Measure	Complexity
Time Complexity	$O(n)$
Space Complexity	$O(1)$

★ Key Takeaway

Sliding Window helps us solve many array / string problems in $O(n)$ time by avoiding unnecessary nested loops.



Common Mistakes

- ✗ Forgetting to move L when condition is violated.
- ✗ Not handling edge cases (empty array, $k > n$).
- ✗ Off-by-one errors in window boundaries.



Best Practices

- ✓ Add new element with R.
- ✓ Check condition and shrink from L if needed.
- ✓ Keep track of the result during each valid window.

@qa_insights

@qa_insights | Quality Insights, Better Careers!

@qa_insights

Prefix Sum & Difference Array

① Prefix Sum

- Prefix Sum (or Cumulative Sum) stores the sum of elements from start till current index.
- Helps to compute range sums in $O(1)$ after $O(n)$ preprocessing.

Example:

Array A = [3, 1, 4, 2, 5]

Index	0	1	2	3	4
A[i]	3	1	4	2	5
Prefix[i]	3	4	8	10	15

Prefix[i] =
Sum of A[0..i]

② Difference Array

- Difference Array stores the difference between elements of the original array.
- Useful for performing range updates efficiently.
- After all updates, we rebuild the original array using prefix sum.

Example:

A = [3, 1, 4, 2, 5]

Index	0	1	2	3	4
A[i]	3	1	4	2	5
Diff[i]	3	-2	3	-2	3

How to build Diff[]

Diff[0] = A[0]
Diff[i] = A[i] - A[i-1]
for i > 0

③ Range Sum Queries

- Find sum of elements in range [L, R] (0-indexed) using Prefix Sum.

Formula:

Sum(L, R) = Prefix[R] - Prefix[L-1]
(if L > 0, else Prefix[R])

Example:

A = [3, 1, 4, 2, 5]

Prefix = [3, 4, 8, 10, 15]

Query: Sum from L = 1 to R = 3
= Prefix[3] - Prefix[0]
= 10 - 3 = 7

④ Applications

- ✓ Range Sum Queries
- ✓ Range Update + Point Query
- ✓ Range Update + Range Query (with Prefix Sum on Difference Array)
- ✓ Subarray Sum Problems
- ✓ Frequency Count in Subarrays
- ✓ 2D Range Sum Queries (using 2D Prefix Sum)

⑤ Difference Array - Range Update Example

- Add value +5 to all elements in range [1, 3] of array A = [3, 1, 4, 2, 5]

Steps:

- Diff[1] += 5
- Diff[4] -= 5 (R+1 index)

Initial Diff from A:

A = [3, 1, 4, 2, 5]

Diff = [3, -2, 3, -2, 3]

Index: 0 1 2 3 4

After Update (+5 to [1,3]):

Diff = [3, 3, 3, -2, -2]

Index: 0 1 2 3 4

Rebuild using Prefix Sum
(Original Array)

A = [3, 6, 9, 7, 5]

⑥ Complexity

Operation	Prefix Sum	Difference Array (Range Update + Point Query)	Difference Array (Range Update + Range Query)
Preprocessing	$O(n)$	$O(n)$	$O(n)$
Range Sum Query	$O(1)$	$O(n)$	$O(1)$
Range Update	—	$O(1)$	$O(1)$
Space Complexity	$O(n)$	$O(n)$	$O(n)$

★ Interview Tip

Always check if the problem has multiple range queries or multiple range updates. It usually hints that Prefix Sum or Difference Array is the optimal approach.



⚠ Common Mistakes

- ✗ Off-by-one errors in Prefix Sum.
- ✗ Forgetting to handle L = 0 case.
- ✗ Not updating (R + 1) in Difference Array.
- ✗ Not rebuilding original array after updates.



Best Practice

Pick Prefix Sum for queries, Difference Array for updates, and combine both when needed.

Hashing

① Hash Table

- A Hash Table stores **key-value** pairs.
- It uses a hash function to map keys to indices in an array.
- Provides fast average case access.

Hash Function Example:

$$h(\text{key}) = \text{key} \% N$$

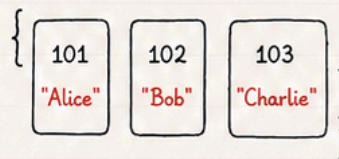
(N = size of table)

Index	Bucket
0	—
1	key2, val2
2	—
3	key1, val1
4	—
5	key3, val3

② HashMap (Key → Value)

- Stores **unique** keys and their values.
- Fast insert, delete and search on average.
- Implemented using Hash Table.

Example:



Java (Concept):

```

HashMap<Integer, String> map
= new HashMap<>();
map.put(101, "Alice");
map.get(101); // "Alice"
map.containsKey(102);
  
```

③ HashSet (Unique Values)

- Stores only **unique** elements.
- Essentially a HashMap with keys only.
- Fast add, remove and contains operations.

Example:

```

{ 10, 20, 30, 40 }
  
```

Java (Concept):

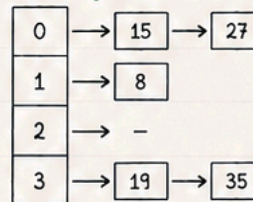
```

HashSet<Integer> set = new
HashSet<>();
set.add(10);
set.contains(20);
set.remove(10);
  
```

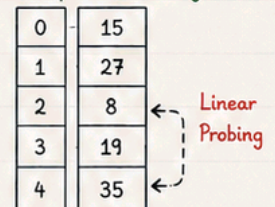
④ Collision

- When two different keys hash to the **same index**.
- Handled using:

1) Chaining (Separate Chaining)



2) Open Addressing



⑤ Load Factor (α)

- $\alpha = \frac{\text{Number of elements}}{\text{Number of buckets}}$
- Helps to measure how full the hash table is.
- When α exceeds a threshold (e.g., 0.75), resize the table (rehashing) to maintain $O(1)$ performance.

Important Note

Keep load factor balanced.

Too high → more collisions.

Too low → wasted space.

⑥ Applications

- ✓ Frequency counting of elements
- ✓ Caching / Memorization
- ✓ Detect duplicates
- ✓ Group anagrams
- ✓ Database indexing
- ✓ Symbol tables in compilers
- ✓ Implement sets and dictionaries

Example: Frequency Count

Array: [4, 1, 2, 4, 2, 1, 4]

Map:

Key	1	2	4
Count	2	2	3

⑦ Complexity (Average Case)

Operation	HashMap	HashSet	Notes
Insert	$O(1)$	$O(1)$	Amortized $O(1)$
Search	$O(1)$	$O(1)$	Fast lookup
Delete	$O(1)$	$O(1)$	If key/element exists
Space	$O(n)$	$O(n)$	n = number of elements

★ Interview Tip

Mention worst case can be $O(n)$ if many collisions happen.



⚠ Common Mistakes

- ✗ Choosing bad hash function.
- ✗ Ignoring load factor.
- ✗ Not handling collisions.
- ✗ Forgetting to rehash when resizing.

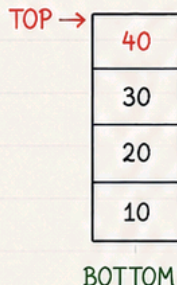
Stack

@qa_insights

@qa_insights

① LIFO (Last In First Out)

- Stack is a linear data structure that follows LIFO principle.
- The element inserted last is the first one to be removed.
- Operations are performed at one end called TOP.



② Operations

Operation	Description	Time Complexity
push(x)	Insert element x at top	O(1)
pop()	Remove top element	O(1)
peek()/top()	Return top element	O(1)
isEmpty()	Check if stack is empty	O(1)
isFull()	Check if stack is full	O(1)

③ Implementation

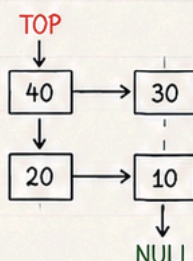
A. Using Array

- Fixed size
- Use an array and a top index to track top element.

```
int stack[MAX];
int top = -1;
push : stack[++top] = x;
pop  : return stack[top--];
```

B. Using Linked List

- Dynamic size
- Use linked list where new nodes are added at head.



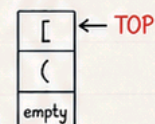
④ Applications

- ✓ Function call management (Call Stack)
- ✓ Expression evaluation (infix, postfix, prefix)
- ✓ Parentheses matching
- ✓ Backtracking (DFS, N-Queens, Maze)
- ✓ Undo/Redo operations in editors
- ✓ Reversing a string or data

Parentheses Matching

Expression: $(a + b) \times [c - d]$

Stack



Balanced



⑤ Complexity

Operation	Time Complexity	Space Complexity
push	O(1)	O(n)
pop	O(1)	O(n)
peek/top	O(1)	O(n)
isEmpty	O(1)	O(n)
isFull	O(1)	O(n)



Important Note

All stack operations are done in constant time O(1) since we only access the top element.

⑥ Interview Problems

1. Valid Parentheses
2. Min Stack
3. Evaluate Reverse Polish Notation
4. Daily Temperatures
5. Next Greater Element (I & II)
6. Largest Rectangle in Histogram
7. Basic Calculator
8. Remove K Digits



Tip

Always think if the problem needs LIFO order or backtracking.



★ Key Takeaway

Stack is simple but powerful!
Remember → Last In,
First Out (LIFO).



Common Mistakes

- ✗ Popping from an empty stack.
- ✗ Not checking boundary conditions.
- ✗ Using wrong data structure.



Best Practices

- ✓ Always check if stack is empty before pop.
- ✓ Keep code clean and modular.
- ✓ Use meaningful variable names (top).

@qa_insights

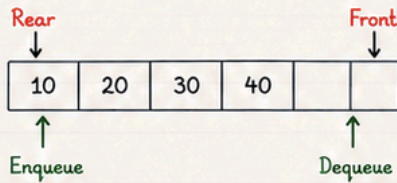
@qa_insights | Quality Insights, Better Careers!

@qa_insights

Queue & Deque

① FIFO (First In First Out)

- Queue follows FIFO principle.
- The element inserted first is removed first.
- One end for insertion (Rear) and one end for deletion (Front).



Enqueue → add element at Rear
Dequeue → remove element from Front

② Queue Types

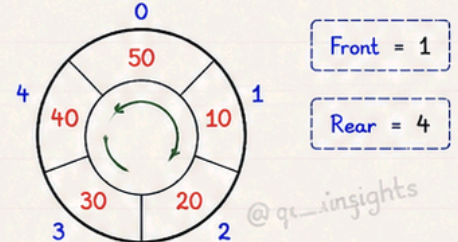
- Simple Queue (Linear Queue)
- Circular Queue
- Priority Queue
- Double Ended Queue (Deque)

★ Remember

Use Queue when order of processing matters.
Use Deque when insertion and deletion are needed from both ends.

③ Circular Queue

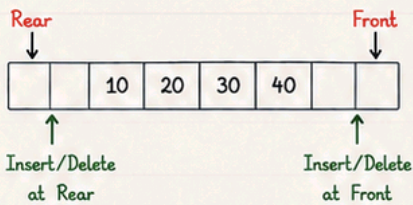
- Last position connects back to first.
- Utilizes all spaces of the array efficiently.



Front = (Front + 1) % N
Rear = (Rear + 1) % N

④ Deque (Double Ended Queue)

- Insertion and Deletion allowed at both ends.
- More flexible than a queue.



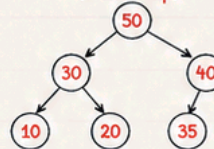
Use Cases

- ✓ Undo/Redo operations
- ✓ Palindrome checking
- ✓ Sliding window maximum (with optimization)

⑤ Priority Queue (Introduction)

- Elements are removed based on priority, not insertion order.
- Highest priority element is removed first (Max-Heap) or lowest (Min-Heap).
- Common implementation: Binary Heap.

Max Priority Queue (Max-Heap)



Min Priority Queue (Min-Heap)



⑥ Complexity

Data Structure	Operation	Time Complexity	Space Complexity	Notes
Simple Queue (Array)	Enqueue	O(1)	O(n)	May need shifting in worst case for linear queue (array full).
	Dequeue	O(1)		
	Peek/Front	O(1)		
Circular Queue (Array)	Enqueue	O(1)	O(n)	Better space utilization using modulo.
	Dequeue	O(1)		
	Peek/Front	O(1)		
Deque (Array)	Insert Front/Rear	O(1)	O(n)	Requires careful index management.
	Delete Front/Rear	O(1)		
	Peek Front/Rear	O(1)		
Priority Queue (Heap)	Insert (push)	O(log n)	O(n)	Implemented using Binary Heap.
	Delete (pop)	O(log n)		
	Peek (top)	O(1)		

💡 Applications

- ✓ Print Queue in operating systems
- ✓ Breadth First Search (BFS)
- ✓ Task scheduling
- ✓ Buffers (I/O)
- ✓ Priority based scheduling

🎯 Interview Problems

1. Implement Queue using Stack
2. Implement Stack using Queue
3. Design Circular Queue
4. Sliding Window Maximum (using Deque)
5. Kth Largest Element in a Stream (using Priority Queue)

★ Key Takeaway

Queue → FIFO order processing.
Deque → Operations from both ends.
Priority Queue → Based on priority (Heap).

⚠ Common Mistakes

- ✗ Confusing Front and Rear.
- ✗ Not handling queue full/empty cases.
- ✗ In circular queue, forgetting modulo.
- ✗ Using wrong heap type in PQ problems.

🔍 Best Practices

- ✓ Always handle edge cases.
- ✓ Use Circular Queue for arrays.
- ✓ Use Deque for optimal sliding window.
- ✓ Choose correct data structure wisely.

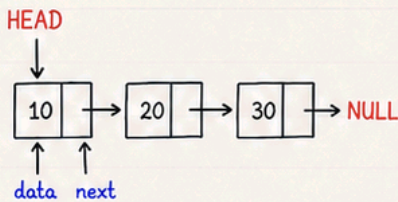
Linked List

@qa_insights

@qa_insights

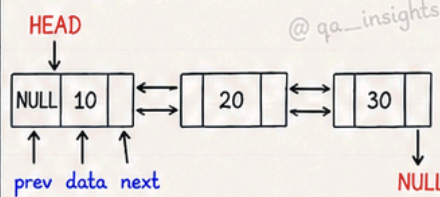
① Singly Linked List

- Each node contains data and a pointer to next node.
- Last node points to NULL.



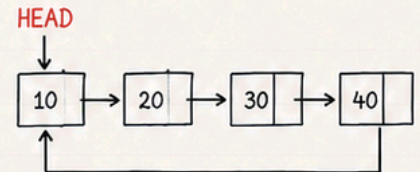
② Doubly Linked List

- Each node contains data, pointer to next and pointer to previous.
- Supports traversal in both directions.



③ Circular Linked List

- Last node points back to the first node.
- No NULL at the end.

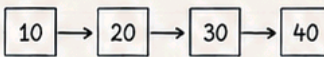


④ Operations

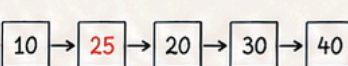
- Insert at Beginning: New node becomes head.
- Insert at End: Traverse to last and add new node.
- Insert at Position: Traverse to (pos-1) and insert.
- Delete at Beginning: Move head to head->next.
- Delete at End: Traverse to second last and delete last.
- Delete at Position: Traverse to (pos-1) and delete next.
- Search: Traverse and compare each node.

Example (Insert 25 at position 2)

Original List:



After Insertion:



💡 Interview Tip

Use a dummy node to handle edge cases (empty list, insert/delete at head) easily.

⑤ Advantages

- ✓ Dynamic size (grows/shrinks at runtime)
- ✓ Efficient insertions and deletions ($O(1)$ with given node)
- ✓ No need of contiguous memory
- ✓ Memory is allocated as needed

★ Best Practice

- ✓ Use meaningful variable names.
- ✓ Handle edge cases (empty list, single node).
- ✓ Free memory in languages like C/C++ to avoid memory leaks.

⚠ Common Mistakes

- ✗ Forgetting to update head pointer.
- ✗ Losing reference to next node during deletion.
- ✗ Not handling NULL conditions.

⑥ Complexity

Operation	Singly Linked List	Doubly Linked List	Circular Linked List
Insert at Head	$O(1)$	$O(1)$	$O(1)$
Insert at Tail	$O(n)$	$O(n)$	$O(n)^*$
Insert at Position	$O(n)$	$O(n)$	$O(n)$
Delete at Head	$O(1)$	$O(1)$	$O(1)$
Delete at Tail	$O(n)$	$O(n)$	$O(n)^*$
Delete at Position	$O(n)$	$O(n)$	$O(n)$
Search	$O(n)$	$O(n)$	$O(n)$
Space Complexity	$O(n)$	$O(n)$	$O(n)$

* $O(1)$ if we maintain a tail pointer (especially for Circular Linked List).

🔑 Key Takeaways

- ✓ Linked List is a linear data structure where nodes are not stored consecutively.
- ✓ Use Singly LL for simple use cases.
- ✓ Use Doubly LL when backward traversal is needed.
- ✓ Use Circular LL for continuous processing (round-robin, queues).
- ✓ Choose the right type based on problem requirements.

@qa_insights

@qa_insights | Quality Insights, Better Careers!

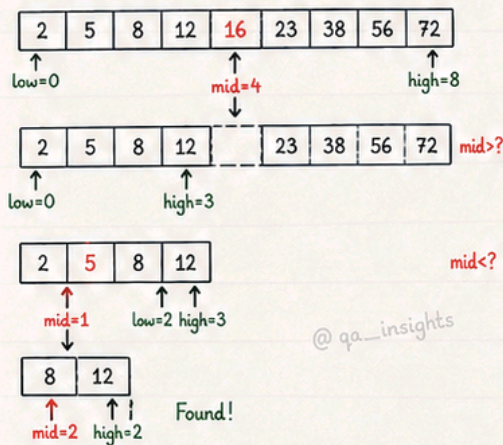
@qa_insights

① Concept

- Binary Search is a **divide and conquer** algorithm.
- It repeatedly divides the sorted array into two halves and decides which half to search.
- Much faster than Linear Search.

Array:	2	5	8	12	16	23	38	56	72
Index :	0	1	2	3	4	5	6	7	8

Target = 16



④ Iterative vs Recursive

Iterative

- Uses loop.
- Space Complexity: $O(1)$.
- Generally preferred.
- Faster in practice.

Recursive

- Uses function calls.
- Space Complexity: $O(\log n)$ (call stack).
- More intuitive for divide & conquer.

⑥ Common Mistakes

✗ Using (low + high) / 2	May cause integer overflow.
✗ Not updating low/high correctly	Leads to infinite loop.
✗ Applying on unsorted data	Results will be incorrect.
✗ Confusing index with value	Return index, not element.
✗ Off-by-one errors (low <= high)	Use correct boundary conditions.
✗ Using on Linked List	Not efficient ($O(n)$).

② Prerequisites

- Array/collection must be sorted (ascending order).
- Random access data structure (like Array, Vector).
- Comparable elements.

☆ Remember

Binary Search works
ONLY on sorted data.
If not sorted, sort first
(sorting cost: $O(n \log n)$).

③ Algorithm (Iterative)

```
int binarySearch(int[] arr, int key){
    int low = 0, high = arr.length - 1;
    while(low <= high){
        int mid = low + (high - low) / 2;
        if(arr[mid] == key) return mid;
        else if(arr[mid] < key) low = mid + 1;
        else high = mid - 1;
    }
    return -1; // not found
}
```

 Interview Fact

Using $\text{low} + (\text{high} - \text{low}) / 2$ prevents integer overflow.

⑤ Complexity

Case	Time Complexity	Space Complexity
Best Case	$O(1)$	$O(1)$ Iterative $O(\log n)$ Recursive
Average Case	$O(\log n)$	$O(1)$ Iterative $O(\log n)$ Recursive
Worst Case	$O(\log n)$	$O(1)$ Iterative $O(\log n)$ Recursive

☆ Key Takeaways

- ✓ Binary Search is $O(\log n)$ – very efficient!
- ✓ Always ensure array is sorted.
- ✓ Eliminates half of the search space in each step.
- ✓ Perfect for large datasets.

 Real World Analogy

Think of searching a word in a dictionary. You check the middle word and decide which half to open.

Bit Manipulation Basics

@qa_insights

① Binary Numbers

- Computers work with 0s and 1s.
- Each bit has a position (index) and value = 2^{index} .
- Example: Decimal 13

Power of 2	2^3	2^2	2^1	2^0
	8	4	2	1
Binary	1	1	0	1

Calculation

$$8 + 4 + 0 + 1 = 13$$

② AND (&)

Result is 1 only if both bits are 1.

A	B	A & B
0	0	0
0	1	0
1	0	0
1	1	1

Example:

A = 12 (1100)

B = 10 (1010)

1100

& 1010

1000 (8)

③ OR (|)

Result is 1 if at least one bit is 1.

A	B	A B
0	0	0
0	1	1
1	0	1
1	1	1

Example:

A = 12 (1100)

B = 10 (1010)

1100

| 1010

1110 (14)

@qa_insights

④ XOR (^)

Result is 1 if bits are different.

A	B	A ^ B
0	0	0
0	1	1
1	0	1
1	1	0

Example:

A = 12 (1100)

B = 10 (1010)

1100

^ 1010

0110 (6)

⑤ NOT (~)

Inverts all bits (0 → 1, 1 → 0).

Works on fixed bit length (e.g., 8 or 32).

Example (8-bit representation)

A = 00001100 (12)

~A = 11110011 (-13 in signed int)



Note:

$$\sim A = -(A + 1)$$

⑥ Left Shift (<<)

Shifts bits left by k positions.

Fills 0s on the right.

Equivalent to multiply by 2^k .

Example: A = 5 (0101)

A << 1 = 1010 (10)

A << 2 = 10100 (20)

5 (0101)

0	1	0	1
---	---	---	---

↓ << 1

1	0	1	0
---	---	---	---

↓ << 2

1	0	1	0	0
---	---	---	---	---

@qa_insights

⑦ Right Shift (>>)

- Shifts bits right by k positions.
- For unsigned: fills 0s on the left.
- For signed: fills sign bit (arithmetic shift).
- Equivalent to divide by 2^k .

Example: A = 20 (10100)

A >> 1 = 01010 (10)

A >> 2 = 00101 (5)

@qa_insights

20 (10100)
1 0 1 0 0
↓ >> 1
0 1 0 1 0
↓ >> 2
0 0 1 0 1

★ Important Note

- Bit operations are extremely fast.
- Used in competitive programming and system level problems.
- Practice to identify patterns!



@qa_insights

⑧ Interview Tricks

Trick	Explanation / Use
Check even / odd	$(n \& 1) == 0 \rightarrow \text{even}, (n \& 1) == 1 \rightarrow \text{odd}$
Multiply by 2	$n \ll 1$
Divide by 2	$n \gg 1$
Power of 2	$n > 0 \text{ and } (n \& (n - 1)) == 0$
Turn off rightmost set bit	$n \& (n - 1)$
Isolate rightmost set bit	$n \& \sim n$ (Two's complement)
Count set bits	Use Brian Kernighan's Algo: $n \& (n - 1)$ in loop
Swap two numbers	$a = a \oplus b; b = a \oplus b; a = a \oplus b;$
Check kth bit	$(n \gg k) \& 1$
Set kth bit	$n (1 \ll k)$
Clear kth bit	$n \& \sim(1 \ll k)$
Toggle kth bit	$n \oplus (1 \ll k)$

@qa_insights

★ Key Takeaway

Bit Manipulation improves performance and helps solve many tricky problems elegantly.



Common Mistake

- Forgetting about negative numbers (two's complement).
- Confusing << and >> behavior for signed integers.



Best Practice

- Understand binary representation.
- Practice bit tricks regularly.
- Use visualisation while learning.
- Recognize patterns in problems.



@qa_insights

@qa_insights | Quality Insights, Better Careers!

@qa_insights

Problem Solving Patterns

① Brute Force

- Try all possible solutions directly.
- Simple to implement.
- Inefficient for large inputs.
- Use when constraints are very small.

Example:

Find max in array:
Check every element.

Time Complexity: $O(n)$

Space Complexity: $O(1)$

@qa_insights

② Optimize

- Improve the brute force solution.
- Remove unnecessary work.
- Use better data structures or algorithms.
- Goal $\rightarrow$ Reduce Time or Space Complexity.

Example:

Use HashMap instead of nested loops.

Time Complexity: $O(n)$

(instead of $O(n^2)$)

③ Pattern Recognition

- Identify the underlying pattern in the problem.
- Map the problem to known patterns.
- Common Patterns:
 - Two Pointer
 - Sliding Window
 - Prefix Sum
 - Fast & Slow Pointer
 - DFS / BFS
 - Binary Search

@qa_insights

④ Dry Run

- Run the algorithm on small test cases.
- Track variables and state changes.
- Helps in understanding and debugging.

Example:

Array: [2, 7, 11, 15]

Target: 9

i	j	Sum	Action
0	3	17	Sum > 9 $\rightarrow$ j--
0	2	13	Sum > 9 $\rightarrow$ j--
0	1	9	Found!

⑤ Edge Cases

- Check boundary conditions.
- Handle empty inputs.
- Single element / duplicates.
- Max / Min values.
- Negative numbers.
- Large input size.



Common Edge Cases

- ✓ Empty array / string
- ✓ Single element
- ✓ All elements same
- ✓ Already sorted / reverse sorted
- ✓ Max / Min integer values

@qa_insights

⑥ Complexity Analysis

- Always analyze Time & Space.
- Helps in choosing the best approach.
- Focus on worst case.

Notation	Name	Growth	Example
$O(1)$	Constant	Best	Access array
$O(\log n)$	Logarithmic	Very Good	Binary Search
$O(n)$	Linear	Good	Single Loop
$O(n \log n)$	Linearithmic	OK	Merge Sort
$O(n^2)$	Quadratic	Bad	Nested Loops
$O(2^n)$	Exponential	Very Bad	Recursion (subset)

⑦ Coding Tips

- ✓ Write clean and readable code.
- ✓ Use meaningful variable names.
- ✓ Break the solution into small functions.
- ✓ Test with multiple cases.
- ✓ Don't over-optimize early.
- ✓ Comment key logic.
- ✓ Handle errors & edge cases.
- ✓ Practice regularly!



Common Problem Solving Flow



Understand
Read problem carefully



Plan
Think of possible approaches



Apply
Implement best approach



Test
Run on test cases & dry run



Optimize
Improve if needed

@qa_insights

★ Key Takeaways

- ✓ Start simple (Brute Force).
- ✓ Look for patterns.
- ✓ Optimize using right techniques.
- ✓ Always analyze complexity.
- ✓ Practice + Consistency = Success!

@qa_insights



Common Mistakes

- ✗ Not reading problem properly.
- ✗ Jumping to code without plan.
- ✗ Ignoring edge cases.
- ✗ Not analyzing complexity.
- ✗ Not testing enough cases.



Remember

There is no single best algorithm.
The best algorithm depends on the problem and constraints.



Interview Tip

Communicate your approach clearly. Explain your thought process. Write correct & bug-free code.

DSA Revision Cheat Sheet

① Arrays

- Fixed size, contiguous memory.
- Random access in $O(1)$.
- Insert/Delete in middle costly $O(n)$.

② Strings " "

- Sequence of characters.
- Immutable in many languages.
- Common ops: compare, search, substring.

③ Searching

- Linear Search - $O(n)$
- Binary Search - $O(\log n)$
- Requires sorted array for Binary Search.

④ Sorting $\updownarrow$

- Bubble, Selection - $O(n^2)$
- Insertion - $O(n^2)$
- Merge, Quick, Heap - $O(n \log n)$

⑤ Recursion

- Function calling itself.
- Needs base case.
- Uses call stack.

⑥ Two Pointers $\longleftrightarrow$

- Use two indices.
- Often on sorted array.
- Solve in $O(n)$ time.

⑦ Sliding Window

- Maintain a window/range.
- Fixed or Variable size.
- Useful for subarray/substring problems.

⑧ Prefix Sum Σ

- Precompute prefix sums.
- Range sum in $O(1)$.
- Great for multiple range queries.

⑨ Hashing

- Map values to indices (Hash).
- HashMap, HashSet.
- Avg ops in $O(1)$.

⑩ Stack

- LIFO (Last In First Out).
- Ops: push, pop, peek.
- Used in recursion, parsing, backtracking.

⑪ Queue

- FIFO (First In First Out).
- Ops: enqueue, dequeue.
- Used in BFS, scheduling, buffers.

⑫ Linked List

- Nodes connected by pointers.
- Types: Singly, Doubly, Circular.
- Insert/Delete in $O(1)$ (given node).

⑬ Binary Search

- Divide & conquer on sorted data.
- Time: $O(\log n)$
- Find, first/last occurrence, lower/upper bound.

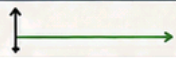
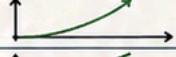
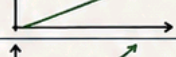
⑭ Bit Manipulation

- Work with bits directly.
- Ops: $\&$, $|$, $\wedge$, $\sim$, $\ll$, $\gg$
- Useful in optimization, XOR tricks, masks.

★ Quick Pattern Map

- Two Pointers $\rightarrow$ Pair / Triplet / Sorted problems
- Sliding Window $\rightarrow$ Subarray / Substring constraints
- Prefix Sum $\rightarrow$ Range Sum Queries
- Hashing $\rightarrow$ Frequency / Uniqueness / Lookup
- Stack $\rightarrow$ Next Greater / Parentheses / Reversal
- BFS (Queue) $\rightarrow$ Shortest path in unweighted graph
- Binary Search $\rightarrow$ Search in sorted / Answer in range

⑮ Common Time Complexities

Notation	Name	Growth	When It Occurs	Examples	Visual
$O(1)$	Constant Time	Best	Accessing array element, hash operations	<code>arr[i]</code> , stack push/pop, <code>hashMap.get()</code>	
$O(\log n)$	Logarithmic Time	Very Good	Binary Search, operations on balanced tree/heap	Binary Search, AVL/Red-Black tree ops	
$O(n)$	Linear Time	Good	Single loop over n elements	Linear Search, traverse list/array	
$O(n \log n)$	Linearithmic Time	Better	Efficient comparison based sorting	Merge Sort, Quick Sort, Heap Sort	
$O(n^2)$	Quadratic Time	Bad	Nested loops on n elements	Bubble Sort, Selection Sort, many brute force solutions	

★ Key Takeaways

- Practice patterns, not just problems.
- Understand Time & Space Complexity.
- Dry run before coding.
- Optimize only after working solution.

⚠ Common Mistakes

- Ignoring edge cases.
- Off-by-one errors.
- Not analyzing complexity.
- Over-complicating simple problems.

💡 Remember

- Every problem has patterns.
- Identify $\rightarrow$ Choose right approach $\rightarrow$ Implement $\rightarrow$ Test $\rightarrow$ Optimize.

🚀 Interview Tip

- Explain your approach clearly, handle edge cases and write clean code!